

HUNGRY DOG

Fuld dokumentation

Produktrapport, procesrapport, logbog og diagrammer

Felt	Indhold
Projekt	Hungry Dog
Produkt	Web/app-løsning til hunde, vægt, måltider, noter og statistik
Teknisk stack	Expo / React Native, TypeScript, .NET Web API, Entity Framework Core og PostgreSQL
Dato	25. maj 2026

Kort resume

Hungry Dog er en enkel og overskuelig app, som jeg har lavet for at give brugeren et sted at holde styr på sine hunde. I appen kan brugeren oprette hunde og følge data som vægt, foder, måltidsstatus og noter.

Dokumentationen beskriver både produktets tekniske opbygning, min arbejdsproces og de vigtigste trin i projektet.

Mit mål har været at lave en løsning, der er realistisk nok til at vise samspillet mellem frontend, backend og database, men stadig overskuelig nok til et skoleprojekt. Derfor har jeg fokuseret på funktioner, som giver mening for brugeren, og på en struktur, som jeg kan forklare tydeligt.

Indhold

- 1. Produktrapport: produkt, teknologivalg, arkitektur, database, API og centrale kodeeksempler.
- 2. Procesrapport: tidsplan, ændringer undervejs, brugerflows, test og refleksion.
- 3. Logbog: kronologisk overblik over projektets 20 arbejdsdage.
- 4. Kort konklusion: samlet vurdering af projektet.

1. Produktrapport

I produktrapporten forklarer jeg, hvad Hungry Dog er, hvilke teknologier jeg har valgt, og hvorfor de passer til formålet. Produktet er bygget som en web/app-løsning med en tydelig opdeling mellem frontend, backend og database.

1.1 Produktbeskrivelse

Hungry Dog hjælper brugeren med at holde styr på en eller flere hunde. Brugeren kan oprette en konto, logge ind, oprette hunde, se hundekort på forsiden, registrere vægt, se statistik, gemme måltidsstatus og skrive noter til den enkelte hund.

- Jeg har brugt login og JWT-token til at adskille brugerdata.
- Hver bruger kan have flere hunde i systemet.
- Hver hund kan have flere vægtmålinger, måltider og noter.
- Statistik-siden bruger dogId, så den viser data for den hund, brugeren har valgt.

1.2 Teknologier valgt til produktet

Teknologi	Brugt til	Hvorfor det passer
Expo / React Native	Frontend og skærme i appen.	Jeg valgte Expo, fordi det er nemt at køre appen på web og giver en enkel struktur til et skoleprojekt.
TypeScript	Frontend-kode og typer.	TypeScript hjælper mig med at holde styr på datatyper, så fejl mellem sider og services bliver nemmere at opdage.
Expo Router	Navigation mellem Login, Forside, Tilføj hund, Statistik og Profil.	Routeren gør skærmene overskuelige og gør det nemt at sende dogId videre til Statistik.
.NET Web API	Backend og API-endpoints.	Jeg valgte .NET, fordi det giver en tydelig opdeling i controllers og services, som er nem at forklare.
Entity Framework Core	Forbindelse mellem C#-modeller og databasen.	EF Core gør det muligt at arbejde med databasen gennem modeller og migrations i stedet for kun rå SQL.
PostgreSQL	Database til brugere, hunde, vægtlogs, måltider og noter.	PostgreSQL passer godt til relationelle data, fordi en bruger kan have flere hunde, og en hund kan have flere logs.
JWT-token	Login og adgang til beskyttede endpoints.	Tokenet gør, at API'et kan finde den bruger, der er logget ind, uden at brugeren skal sende email og password hver gang.
DBeaver	Visning og kontrol af databasen.	Jeg brugte DBeaver til at kontrollere tabeller og data visuelt under udviklingen.

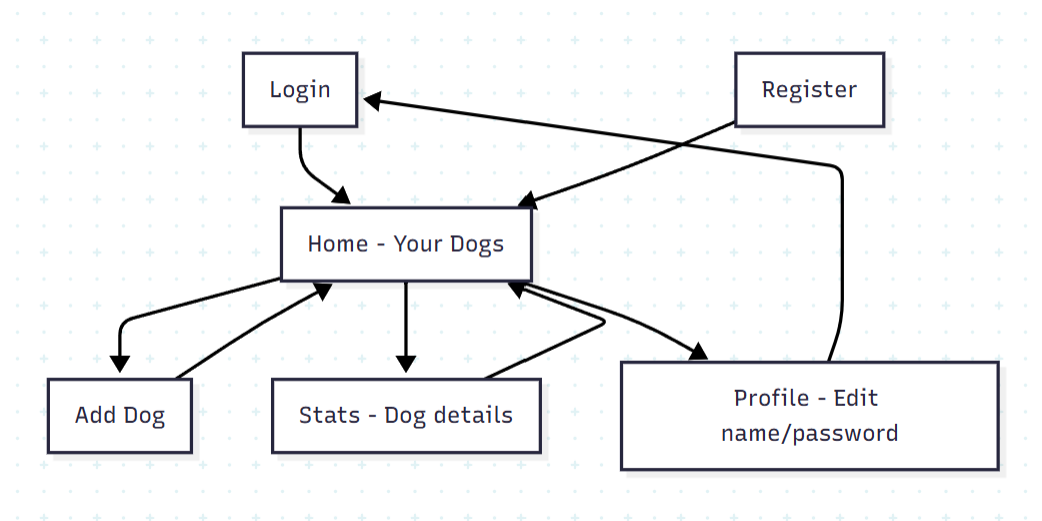
1.3 Systemdiagram

Systemdiagrammet viser den overordnede opbygning. Brugeren anvender appen i browseren, frontend sender forespørgsler til API'et, og API'et gemmer eller henter data i databasen.



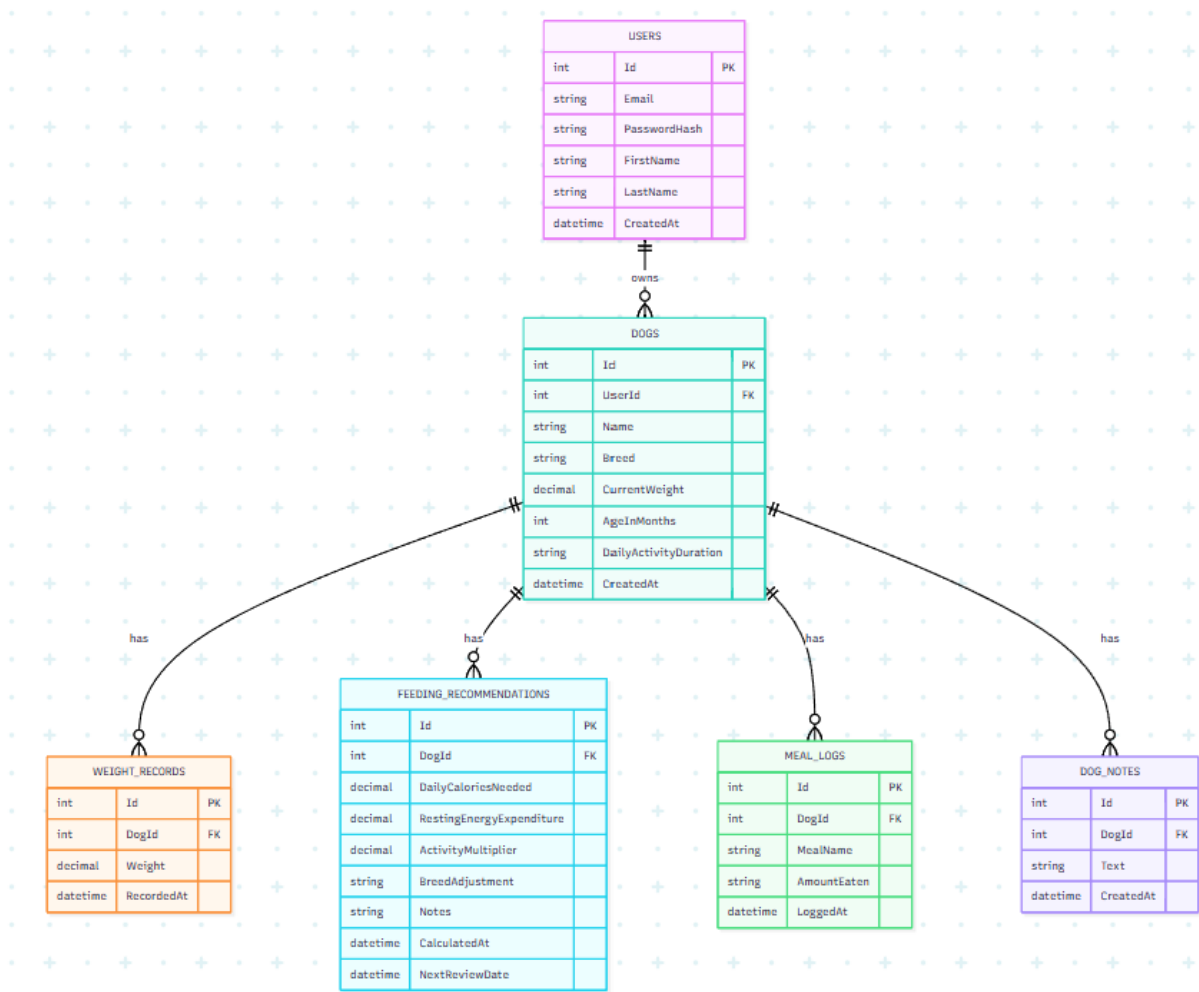
1.4 Frontend-skærmforløb

Frontend er opbygget omkring login, forside, oprettelse af hund, statistik og profil. Jeg har lavet flowet sådan, at brugeren hurtigt kan komme fra forsiden til den valgte hunds statistik. Knappen Vis detaljer sender hundens id videre til statistik-siden.



1.5 Database ERD

Databasen er relationel. Jeg har valgt denne struktur, fordi en bruger kan eje flere hunde, og hver hund kan have flere vægtmålinger, måltidslogs, noter og foderanbefalinger. Når en hund slettes, slettes de relaterede data også via cascade delete.



Tabel	Bruges til
Users	Gemmer brugerens navn, email og password-hash.
Dogs	Gemmer hundens navn, race, alder, vægt, aktivitetsniveau og UserId.
WeightRecords	Gemmer historik over hundens vægt, så grafen kan vises.
MealLogs	Gemmer måltidsnavn og om hunden spiste alt, halvt, lidt eller ingenting.
DogNotes	Gemmer noter, som brugeren selv skriver om hunden.
FeedingRecommendations	Gemmer anbefalet fodermængde, RER, aktivitetsfaktor og næste review-dato.

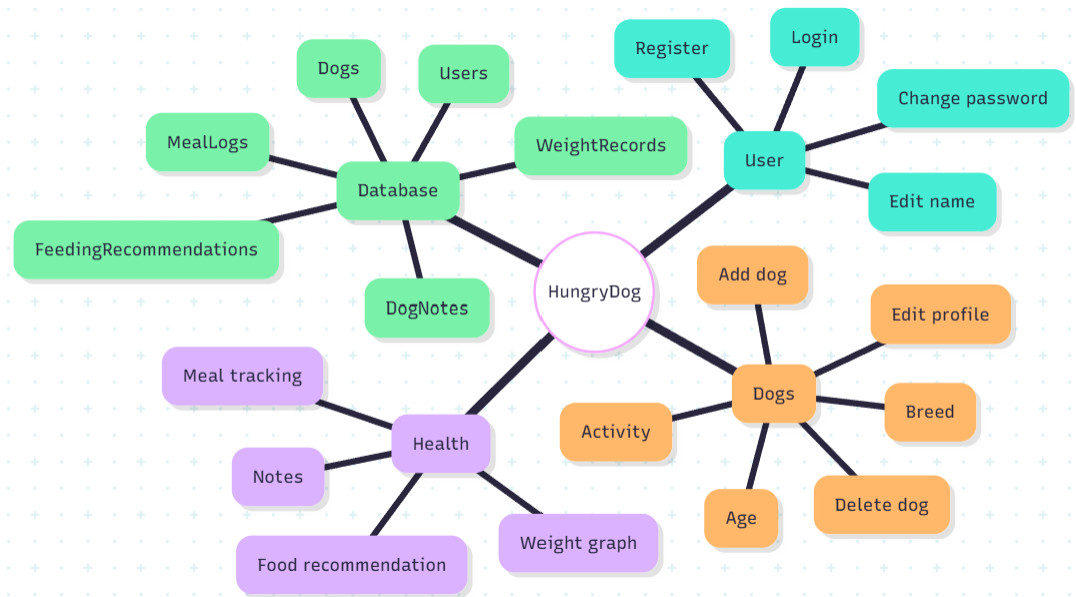
1.6 API-endpointdiagram

API’et fungerer som mellemed mellem appen og databasen. Frontend kalder endpoints, og backend sørger for validering, login, ejerskabstjek og databasearbejde.

Område	Endpoints	Formål
Auth	/api/auth/register, /api/auth/login, /api/auth/me, /api/auth/profile, /api/auth/password	Opret bruger, login, hent profil, rediger profil og skift password.
Dogs	/api/dogs, /api/dogs/{id}	Opret, hent, opdater og slet hunde for den indloggede bruger.
Weight	/api/weight, /api/weight/{dogId}/history, /api/weight/{dogId}/trend, /api/weight/{dogId}/status	Gem og hent vægtmålinger samt beregn simpel trend og status.
Feeding	/api/feeding/calculate, /api/feeding/{dogId}/latest, /api/feeding/{dogId}/details, /api/feeding/{dogId}/history	Beregner og henter foderanbefalinger.
Meal logs	/api/feeding/meal-log, /api/feeding/{dogId}/meal-log/latest, /api/feeding/{dogId}/meal-log/recent	Gemmer og henter måltidsstatus.
Notes	/api/dogs/{dogId}/notes, /api/dogs/{dogId}/notes/{noteId}	Opret, hent og slet noter for en hund.

1.7 Funktionsoversigt

- Brugeren kan oprette konto og logge ind.
- Brugeren kan oprette hunde med navn, race, alder, vægt og aktivitet.
- Forsiden viser brugerens hunde som kort med knapper til detaljer og noter.
- Statistik-siden viser vægt, alder, aktivitet, race, madstatus og anbefalinger.
- Brugeren kan logge ny vægt, ændre hundens profil og slette hunden.
- Brugeren kan skrive og slette noter til hver hund.



1.8 Centrale kodeeksempler

Kodeeksemplerne viser de vigtigste tekniske valg i projektet. Jeg har valgt at tage dem med, fordi de viser, hvordan login, sikkerhed, frontend-kald og relationer i databasen hænger sammen.

Kodeeksempel	Hvad det viser	Hvorfor det er med
JWT-token	Backend opretter et token med brugerens id og email.	Det viser, hvordan login hænger sammen med beskyttede endpoints.
Authorization-header	Frontend sender token med ved API-kald.	Det viser forbindelsen mellem frontend og backend.
SHA256 password-hash	Password laves om til en hash, før det gemmes.	Det viser, at passwords ikke gemmes som ren tekst.
Entity Framework-relation	Dog kobles til User i databasen.	Det viser, hvorfor databasen passer til relationelle data.

TokenService.cs - oprettelse af JWT-token

```
var claims = new[]
{
    new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
    new Claim(ClaimTypes.Email, user.Email),
    new Claim(ClaimTypes.Name, $"{user.FirstName} {user.LastName}")
};

var token = new JwtSecurityToken(
    issuer: _config["Jwt:Issuer"] ?? "HungryDogApi",
    audience: _config["Jwt:Audience"] ?? "HungryDogClient",
    claims: claims,
    expires: DateTime.UtcNow.AddHours(24),
    signingCredentials: credentials
);
```

Jeg har brugt JWT, fordi API'et skal kunne se, hvilken bruger der er logget ind. Brugerens id ligger i tokenet og bruges til at hente de rigtige hunde.

api.ts - frontend sender token med til backend

```
headers: {
    'Content-Type': 'application/json',
    ...(token ? { Authorization: `Bearer ${token}` } : {}),
    ...(options.headers || {}),
}
```

Frontend gemmer token efter login og sender det med i Authorization-headeren. Det gør, at backend kan beskytte endpoints med [Authorize].

AuthService.cs - SHA256 hash af password

```
private static string HashPassword(string password)
{
    using (var sha256 = SHA256.Create())
    {
        var hashedBytes = sha256.ComputeHash(Encoding.UTF8.GetBytes(password));
        return Convert.ToBase64String(hashedBytes);
    }
}
```

I koden bruger jeg SHA256. Det betyder, at adgangskoden ikke gemmes som almindelig tekst i databasen, men som en hash. Til et skoleprojekt er det simpelt at forklare, men i produktion ville BCrypt eller ASP.NET Identity være bedre.

AppDbContext.cs - relation mellem User og Dog

```
modelBuilder.Entity<Dog>()
    .HasOne<User>()
    .WithMany()
    .HasForeignKey(d => d.UserId)
    .OnDelete(DeleteBehavior.Cascade);
```

Denne relation viser, at en hund hører til en bruger. Det passer til formålet, fordi hver bruger kun skal se sine egne hunde.

2. Procesrapport

Procesrapporten beskriver, hvordan jeg planlagde projektet, hvordan arbejdet faktisk udviklede sig, og hvilke tekniske valg og ændringer jeg lavede undervejs.

2.1 Tidsplan og prioritering

Jeg prioriterede projektet sådan, at de vigtigste funktioner kom til at virke først. Først skulle login, database, hunde og statistik hænge sammen. Derefter arbejdede jeg med ekstra funktioner, kvalitet og dokumentation.

Fase	Planlagt arbejde	Formål
1. Grundstruktur	Opsætte Expo frontend, .NET API og database.	Få projektets dele til at kunne køre sammen.
2. Login	Opret bruger, login og token.	Sikre at data hører til den rigtige bruger.
3. Hunde	Opret, vis, rediger og slet hunde.	Få appens vigtigste data til at virke.
4. Statistik	Vægtgraf, aktivitet, måltider og anbefalinger.	Gøre appen mere brugbar for brugeren.
5. Ekstra funktioner	Noter, profilredigering og diagrammer.	Gøre projektet mere komplet til aflevering.
6. Dokumentation	Produktrapport, procesrapport, logbog og diagrammer.	Forklare valg, proces og opbygning.

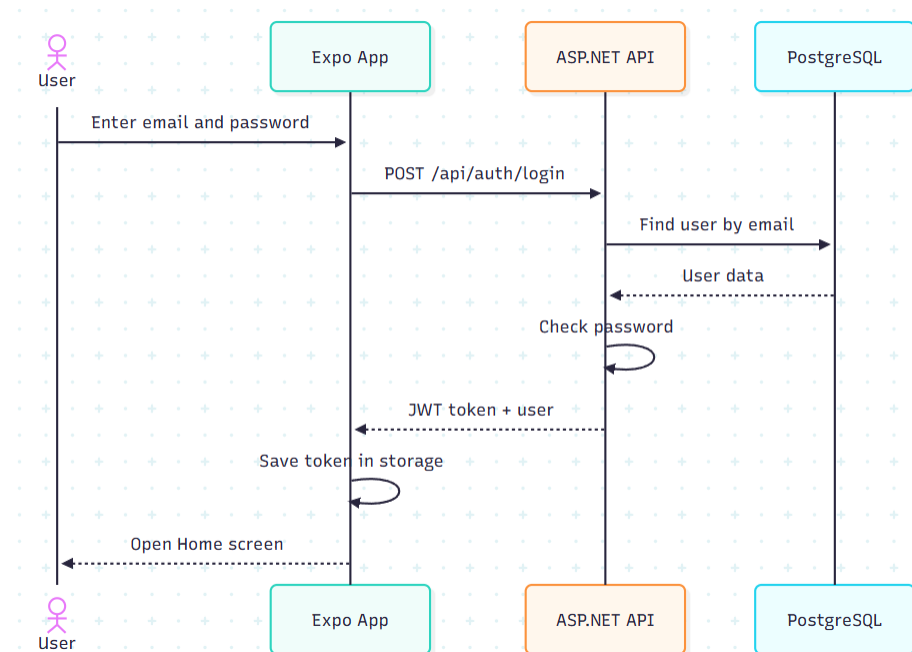
2.2 Hvordan processen har været

Processen var ikke helt lineær. Flere problemer viste sig først, da frontend, backend og database blev koblet sammen. Derfor arbejdede jeg meget iterativt: jeg lavede en funktion, testede den, rettede fejl og fortsatte derefter med næste del.

Problem / ændring	Hvad der skete	Løsning / næste skridt
Databasevalg	Jeg startede med PostgreSQL, men fik problemer med opsætningen og adgangskoden. Jeg prøvede også at kigge på Docker og Supabase som mulige løsninger, men valgte til sidst at få nulstillet min PostgreSQL-adgangskode og fortsætte lokalt.	Connection string blev sat til lokal PostgreSQL, så projektet kunne køre stabilt og simpelt.
Expo web storage	SecureStore virkede ikke korrekt i web-browseren, selvom det gav mening på native.	Jeg lavede en storage-service, som bruger localStorage på web og SecureStore på native.
Opret hund	Hunden blev oprettet, men forsiden viste ikke altid tydeligt den nye hund med det samme.	Efter oprettelse sendes brugeren tilbage til Forside, og listen hentes igen.
Statistik for valgt hund	Statistik skulle ikke vise tilfældige data, men data for den hund brugeren havde valgt.	Jeg brugte dogId i navigationen, så Statistik henter korrekt hund.
Måltider og noter	Appen manglede mere brugbar daglig data, så den ikke kun handlede om vægt.	Jeg tilføjede MealLogs og DogNotes, så brugeren kan registrere spisning og skrive huskenoter.

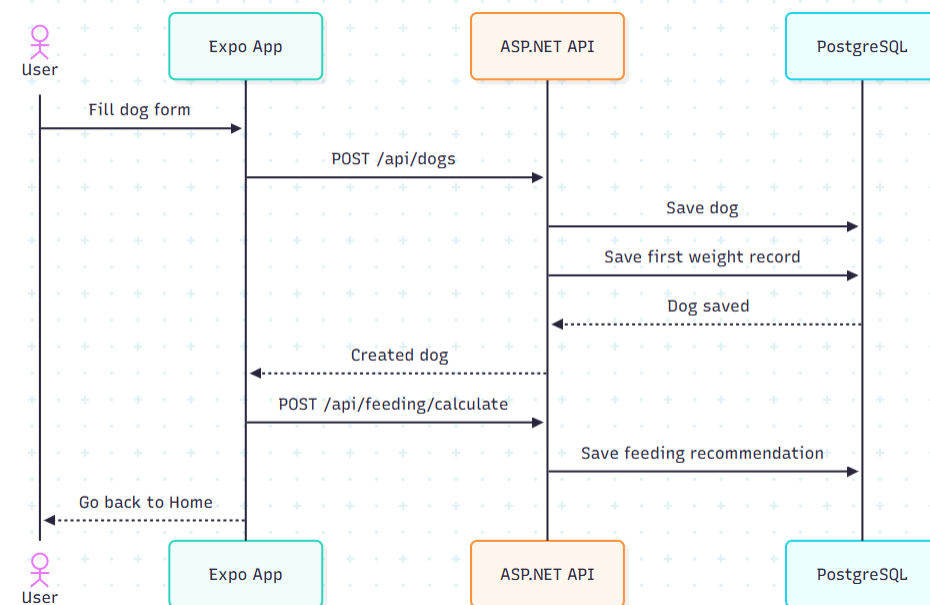
2.3 Login-flow

Login-flowet viser, hvad der sker, når brugeren logger ind. Tokenet gemmes i frontend, så appen kan kalde beskyttede API-endpoints uden at brugeren skal logge ind igen for hvert kald.



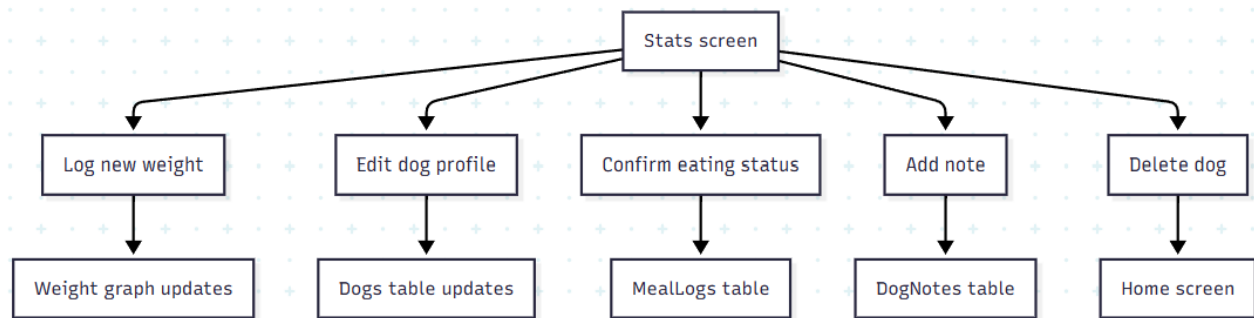
2.4 Tilføj hund-flow

Tilføj hund-flowet viser processen, når brugeren opretter en hund. Når hunden er gemt, sendes brugeren tilbage til forsiden, hvor den nye hund vises.



2.5 Statistik-flow

Statistik-flowet viser, hvordan brugeren ser og ændrer data for en bestemt hund. Jeg bruger dogId til at sikre, at siden henter den rigtige hunds data.



2.6 Data input og validering

Appen modtager data fra brugeren gennem formularer og knapper. Frontend kontrollerer de vigtigste felter, mens backend tjekker ejerskab, datatyper og om hunden findes for den indloggede bruger.

Funktion	Input	Kontrol
Login	Email og password	Backend tjekker bruger og password-hash.
Opret hund	Navn, race, alder, vægt og aktivitet	Backend gemmer hunden på brugerens UserId.
Log vægt	DogId, vægt og tidspunkt	Vægt skal være over 0, og hunden skal tilhøre brugeren.
Måltid	DogId, måltidsnavn og mængde spist	Backend normaliserer Breakfast/Dinner og All/Half/Small/None.
Noter	DogId og fritext	Tomme noter afvises, og tekst trimmes, før den gemmes.

2.7 Valg og begrundelser undervejs

Valg	Begrundelse
Expo frontend	Jeg valgte Expo, fordi det er nemt at lave en simpel web/app-løsning med flere skærme.
.NET Web API	Jeg valgte .NET, fordi det giver en tydelig backend med controllers, services og databaseadgang.
PostgreSQL	Jeg valgte PostgreSQL, fordi databasen er realistisk, men stadig simpel nok til skoleprojektet.
Separate services	Jeg delte koden op i services, så login, hunde, vægt og foder blev lettere at forstå.
Simple datafelter	Jeg fokuserede på information, som brugeren faktisk kan indtaste hurtigt.

2.8 Test og kvalitet

- Jeg byggede backend med dotnet build for at sikre, at API'et kunne kompilere.
- Jeg opdaterede databasen med migrations, så tabellerne passede til modellerne.
- Jeg testede frontend ved at køre Expo web og åbne appen på localhost.
- Jeg testede de vigtigste brugerflows: login, opret hund, vis hunde på Forside og vis statistik for valgt hund.
- Jeg testede sikkerheden ved at bruge [Authorize] på endpoints og UserId fra JWT-token.

2.9 Evaluering af processen

Den vigtigste erfaring fra projektet er, at samspillet mellem frontend, backend og database først bliver tydeligt, når alle dele kobles sammen. Jeg oplevede flere gange, at noget så rigtigt ud i koden, men først virkede helt korrekt, når jeg testede det i appen. Derfor var det nødvendigt at arbejde iterativt og løbende rette dataflowet.

3. Logbog

Logbogen er skrevet kort og kronologisk. Den viser, hvad jeg arbejdede med på de 20 vigtigste arbejdsdage i projektet. Den er ikke skrevet som en detaljeret dagbog, men som et overblik over min proces og mine vigtigste fremskridt.

Dato / dag	Hvad jeg lavede
Dag 1	Jeg oprettede projektet og fik den første Expo frontend og .NET Web API sat op. Jeg valgte også, at projektet skulle hedde Hungry Dog.
Dag 2	Jeg lavede den første grundstruktur i frontend med mapper, skærme og navigation, så appen havde en tydelig opbygning.
Dag 3	Jeg begyndte på backend-strukturen med controllers, services og modeller. Jeg satte også de første database-modeller op.
Dag 4	Jeg arbejdede med PostgreSQL og fik databasen koblet til projektet med Entity Framework Core. Jeg havde lidt problemer med adgangskoden, men fik det løst.
Dag 5	Jeg lavede register-funktionen, så en bruger kan oprette sig i systemet. Jeg testede også, at brugeren blev gemt korrekt i databasen.
Dag 6	Jeg lavede login-funktionen og begyndte at arbejde med JWT-token, så brugeren kunne identificeres efter login.
Dag 7	Jeg fik frontend til at gemme tokenet efter login. Jeg lavede også en løsning, der virker på web med localStorage.
Dag 8	Jeg lavede de første hunde-funktioner i backend: opret, hent, opdater og slet hund. Jeg koblede hundene til den indloggede bruger.
Dag 9	Jeg lavede forsiden i frontend, hvor brugerens hunde bliver vist som kort. Jeg testede, at kun brugerens egne hunde blev vist.
Dag 10	Jeg lavede skærmen til at tilføje en hund. Efter oprettelse sendes brugeren tilbage til forsiden, så den nye hund kan ses.
Dag 11	Jeg begyndte på statistik-siden og fik dogId sendt med i navigationen, så siden kan hente den rigtige hund.
Dag 12	Jeg arbejdede med vægtdata og vægtlogging, så brugeren kan gemme ny vægt og senere se historik.
Dag 13	Jeg lavede mere indhold på statistik-siden, blandt andet alder, race, aktivitet og vægtinformation.
Dag 14	Jeg arbejdede med foderberegning og anbefalinger, så appen kunne give et mere brugbart output til brugeren.
Dag 15	Jeg tilføjede måltidsstatus med Breakfast/Dinner og All/Half/Small/None, så brugeren kan registrere, hvor meget hunden har spist.
Dag 16	Jeg lavede noter til hver hund, så brugeren kan skrive og slette små huskenoter. Det gjorde appen mere praktisk i hverdagen.
Dag 17	Jeg lavede profilredigering og password-skift for brugeren. Jeg testede også, at ændringerne blev gemt korrekt.
Dag 18	Jeg rettede fejl i dataflow mellem frontend, backend og database. Jeg brugte især tid på at teste login, hunde og statistik sammen.
Dag 19	Jeg lavede diagrammer til dokumentationen, blandt andet systemdiagram, frontend-flow, ERD, API-diagram og procesflows.
Dag 20	Jeg samlede dokumentationen, rettede sproget, satte billeder/diagrammer ind og gjorde rapporten klar til aflevering.

4. Kort konklusion

Hungry Dog er en simpel app til at holde styr på hunde, vægt, måltider, noter og anbefalinger. Projektet viser samspillet mellem frontend, backend og database på en overskuelig måde. Jeg synes, løsningen passer godt til et skoleprojekt, fordi funktionerne er praktiske, dataene er simple, og systemets dele er tydeligt adskilt.

Den færdige løsning viser samtidig flere centrale tekniske emner: login med JWT, beskyttede endpoints, relationelle databaser, migrations, services, controllers, frontend-navigation og dataflow mellem app og API. Projektet har også givet mig en bedre forståelse for, hvor vigtigt det er at teste hele flowet samlet og ikke kun kigge på de enkelte dele hver for sig.